

Lezione 16 - Python e OOP - Avanzato

COPIA DI OGGETTI

Problema

 quando facciamo:

```
b = a
```

 NON stiamo copiando l'oggetto

 stiamo copiando:

 il riferimento

ESEMPIO

Codice:

```
from copy import copy, deepcopy
```

```
class Person:
```

```
    def __init__(self, name, age, cats):  
        self.name = name  
        self.age = age  
        self.cats = cats
```

```
a = Person("Giangianni", 32, ["Attila", "Tigre"])
```

```
b = a
```

◆ Cosa succede

👉 a e b :

- puntano allo stesso oggetto

✳️ modificare uno modifica anche l'altro

📌 COPY (copia superficiale)



Codice:

```
c = copy(a)
```

◆ Significato

👉 crea un nuovo oggetto

👉 MA:

- gli attributi interni sono condivisi

✳️ strutture annidate NON vengono copiate

📌 DEEPCOPY (copia completa)



Codice:

```
d = deepcopy(a)
```

◆ Significato

👉 crea:

- nuovo oggetto
- copie indipendenti di tutti gli attributi

✳️ nessuna condivisione

📌 CONFRONTO

Operazione	Risultato
<code>b = a</code>	stesso oggetto
<code>copy(a)</code>	copia superficiale
<code>deepcopy(a)</code>	copia completa

📌 IDEA IMPORTANTE

👉 assegnamento $\neq$ copia

✳️ errore molto comune

Ora che abbiamo capito come vengono gestiti gli oggetti in memoria, passiamo a un problema diverso legato alla struttura del codice: la **duplicazione di codice**

📌 PROBLEMA: DUPLICAZIONE DI CODICE

◆ Idea

👉 spesso creiamo classi diverse che fanno quasi le stesse cose

👉 esempio:

- Gatto
- Coniglio

ESEMPIO

Codice:

```
class Gatto:

    common_name = "Gatto"
    calories_needed = 300

    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.happiness = 0
        self.calories = 0

    def eat(self, food):
        self.calories += food.calories
        if (self.calories > self.calories_needed):
            print("Ho mangiato troppo")
            self.happiness -= 1

    def interact_with(self, animal):
        self.happiness += 1
        print(f"Sto giocando con {animal.name}")
```



Codice:

```
class Coniglio:

    common_name = "Coniglio"
    calories_needed = 200

    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.happiness = 0
        self.calories = 0

    def eat(self, food):
        self.calories += food.calories
        if (self.calories > self.calories_needed):
            print("Ho mangiato troppo")
            self.happiness -= 1

    def interact_with(self, animal):
        self.happiness += 1
        print(f"Sto giocando con {animal.name}")
```



COSA NOTARE

👉 le due classi:

- hanno gli stessi attributi
- hanno gli stessi metodi

👉 cambia solo:

- common_name
- calories_needed

PROBLEMA

👉 il codice è:

✳ duplicato

👉 conseguenze:

- modifiche da fare in più punti
- rischio errori
- codice poco manutenibile

IDEA

👉 serve un modo per:

- scrivere il codice comune una sola volta
- riutilizzarlo

Per risolvere la duplicazione di codice introduciamo un concetto fondamentale dell'OOP: l'**ereditarietà**

EREDITARIETÀ

◆ Idea

👉 invece di duplicare il codice:

- creiamo una classe generale
- le altre classi ereditano da essa

✳ riuso del codice



CLASSE BASE



Codice:

```
class Animale:

    common_name = "Animale"
    calories_needed = 0

    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.happiness = 0
        self.calories = 0

    def eat(self, food):
        self.calories += food.calories
        if (self.calories > self.calories_needed):
            print("Ho mangiato troppo")
            self.happiness -= 1

    def interact_with(self, animal):
        self.happiness += 1
        print(f"Sto giocando con {animal.name}")

    def __str__(self):
        return f"Sono un {self.common_name} di nome {self.name}"
```

CLASSI DERIVATE

Codice:

```
class Gatto(Animale):
```

```
    common_name = "Gatto"  
    calories_needed = 300
```

```
class Coniglio(Animale):
```

```
    common_name = "Coniglio"  
    calories_needed = 200
```

COSA SUCCEDDE

 Gatto e Coniglio:

- ereditano attributi e metodi da Animale
- NON devono riscriverli

 quindi possono usare:

- eat()
- interact_with()
- __init__()

SINTASSI

 ereditarietà:

```
class Figlia(Padre):
```


VANTAGGI

👉 otteniamo:

- meno codice
- maggiore riutilizzo
- più facilità di modifica

✳️ modificare la classe base aggiorna tutto

IDEA IMPORTANTE

👉 la classe base:

- contiene comportamento comune

👉 le classi derivate:

- specializzano

Ora che abbiamo una struttura comune, vediamo come possiamo modificare il comportamento nelle classi derivate con l'**Overriding**

OVERRIDING DEI METODI

◆ Idea

👉 una classe derivata può:

- ridefinire un metodo della classe base

✳️ modificare il comportamento

ESEMPIO

Codice:

```
class Panda(Animale):  
  
    common_name = "Panda"  
    calories_needed = 1100  
  
    def interact_with(self, animal):  
        self.happiness -= 1  
        print(f"Sono un panda solitario, non voglio giocare con {animal.name}")
```

COSA SUCCEDE

👉 Panda :

- ha un metodo `interact_with()` diverso
- NON usa più quello di `Animale`

👉 quando chiamiamo:

```
panda.interact_with(...)
```

👉 viene usato:

✳ il metodo della classe derivata

SIGNIFICATO

👉 overriding significa:

- sostituire un metodo ereditato

VANTAGGIO

👉 possiamo:

- mantenere struttura comune
- ma cambiare comportamenti specifici

IDEA IMPORTANTE

👉 una classe derivata:

- può modificare ciò che eredita

✳️ senza riscrivere tutto

Abbiamo visto che possiamo sostituire completamente un metodo, ma a volte vogliamo modificarlo senza perdere quello della classe base. Introduciamo il metodo **Super**

SUPER()

◆ Idea

👉 a volte vogliamo:

- modificare un metodo
- ma usare anche quello della classe base

👉 per farlo usiamo:

✳️ `super()`

ESEMPIO

Codice:

```
class Carnivoro(Animale):  
  
    def eat(self, food):  
        if food.category == "carne":  
            super().eat(food)  
        else:  
            print(f"Sono un carnivoro, non posso mangiare {food.name}")
```

Codice:

```
class Erbivoro(Animale):  
  
    def eat(self, food):  
        if food.category == "vegetale":  
            super().eat(food)  
        else:  
            print(f"Sono un erbivoro, non posso mangiare {food.name}")
```

COSA SUCCEDE

👉 eat() :

- viene modificato
- ma non viene riscritto completamente

👉 super().eat(food) :

- chiama il metodo della classe base (Animale)

VANTAGGIO

👉 possiamo:

- estendere il comportamento
- riutilizzare codice esistente

✳️ senza duplicazione

IDEA IMPORTANTE

👉 `super()` :

- permette di combinare ereditarietà + overriding

✳️ concetto chiave in OOP

Ora che sappiamo combinare più comportamenti, vediamo cosa succede quando una classe eredita da più classi contemporaneamente. Introduciamo l'**eredità multipla**

EREDITARIETÀ MULTIPLA

◆ Idea

👉 una classe può ereditare da più classi

✳️ non solo da una

ESEMPIO

Codice:

```
class AnimaleDomestico(Animale):  
  
    def pet(self):  
        self.happiness += 1  
        print("Mi stanno coccolando")
```

Codice:

```
class Gatto(Carnivoro, AnimaleDomestico):  
  
    common_name = "Gatto"  
    calories_needed = 300
```

Codice:

```
class Coniglio(Erbivoro, AnimaleDomestico):  
  
    common_name = "Coniglio"  
    calories_needed = 200
```

COSA SUCCEDE

 Gatto e Coniglio :

- ereditano da più classi
- combinano comportamenti

👉 esempio:

- `eat()` → da `Carnivoro` / `Erbivoro`
- `pet()` → da `AnimaleDomestico`

📌 VANTAGGIO

👉 possiamo:

- combinare funzionalità
- evitare duplicazione

✳️ codice più modulare

📌 PROBLEMA (IMPORTANTISSIMO)

👉 se due classi hanno lo stesso metodo:

✳️ quale viene usato?

📌 ORDINE DI RISOLUZIONE (MRO)

👉 Python segue un ordine:

- da sinistra a destra
- secondo la gerarchia

👉 esempio:

```
class D(C, B):
```

👉 prima cerca in:

1. c

2. B

IDEA IMPORTANTE

👉 ereditarietà multipla:

- potente
- ma può creare ambiguità

✳️ va usata con attenzione

Ora che abbiamo visto come funziona l'ereditarietà anche nei casi più complessi, passiamo a un nuovo concetto fondamentale: **Le eccezioni**

ECCEZIONI

◆ Idea

👉 durante l'esecuzione di un programma possono verificarsi errori

👉 invece di far crashare il programma:

✳️ possiamo gestirli

TRY / EXCEPT

Codice:

```
d = {}  
  
try:  
    d[10] = d[10] + 1  
except KeyError:  
    d[10] = 0
```

◆ Spiegazione

👉 se `d[10]` non esiste:

- viene sollevata un'eccezione (`KeyError`)

👉 invece di fermarsi:

- il programma entra nel blocco `except`

✳ errore gestito

PROPAGAZIONE DELLE ECCEZIONI

Codice:

```
def g(d1, d2, x):  
    try:  
        d1[x] = d2[x]  
    except KeyError:  
        print("Catturata l'eccezione in g")
```

```
def f(d1, d2, x):  
    try:  
        g(d1, d2, x)  
    except KeyError:  
        print("Catturata l'eccezione in f")
```

◆ Spiegazione

👉 le eccezioni:

- possono essere catturate a diversi livelli
- si propagano verso l'alto

✳ se non catturate → arrivano al livello superiore

GERARCHIA DELLE ECCEZIONI

Codice:

```
class A(Exception):  
    pass  
  
class B(A):  
    pass  
  
try:  
    raise B()  
except A:  
    print("Catturata A")  
except B:  
    print("Catturata B")
```

◆ Spiegazione

👉 B eredita da A

👉 quindi:

- except A cattura anche B

✳ ordine importante

ECCEZIONI CUSTOM

Codice:

```
class EmptyQueueError(Exception):  
    pass
```



Codice:

```
class Queue:

    def __init__(self):
        self.data = []

    def add(self, x):
        self.data.append(x)

    def remove(self):
        if self.data == []:
            raise EmptyQueueError()
        x = self.data[0]
        self.data = self.data[1:]
        return x
```

◆ Spiegazione

👉 possiamo creare:

- eccezioni personalizzate

👉 e usarle con:

```
raise EmptyQueueError()
```

GESTIONE

Codice:

```
try:
    q1.remove()
except EmptyQueueError:
    pass
```

◆ Idea importante

👉 le eccezioni permettono di:

- controllare gli errori
- evitare crash

✳ rendono il codice più robusto